Spring Framework & Microservices Guide

Comprehensive Hands-on Manual: IoC, DI, Spring Boot, JPA, Hibernate, and Eureka Service Discovery

Focus Areas: Spring IoC, DI (Setter/Constructor), Spring Boot REST API, JPA Query Methods, O/R Mapping, HQL/Native SQL, Eureka Discovery

Target Target: Java Enterprise Architects & Spring Developers

Module 1: Core Spring Framework & IoC Container

Theory Deep Dive: Inversion of Control (IoC) & Dependency Injection (DI)

Inversion of Control (IoC) is a software engineering design principle where the control of object creation, life cycle management, and dependency binding is transferred from application code to a dedicated container or framework. In traditional procedural programming, application code directly instantiates dependencies using the new operator. In Spring's IoC paradigm, the Spring Container manages object instantiation, wiring, and destruction.

BeanFactory vs. ApplicationContext: `BeanFactory` provides the basic DI framework and lazy bean initialization. `ApplicationContext` extends `BeanFactory` to provide enterprise features like event publication, internationalization (i18n), and automatic `BeanPostProcessor` registration.

Exercise 5: Configuring the Spring IoC Container

Spring IoC containers can be configured using three primary approaches: XML Configuration, Annotation-based Configuration, and Java-based Configuration (`@Configuration`). Below is a comparison and complete code implementation for each style.

Configuration Style	Mechanism	Primary Advantages	Recommended Use Case
XML Configuration	External `beans.xml` file	Decouples Java code from configuration; zero recompilation for config changes.	Legacy systems, environment-specific infrastructure beans.
Annotation Configuration	Class-level annotations (`@Component`, `@Autowired`)	High developer velocity; explicit metadata directly near the code.	Standard application components (Controllers, Services, Repositories).
Java-based Configuration	Classes annotated with `@Configuration` & `@Bean`	Type-safe, refactoring-friendly, supports complex initialization logic.	Modern Spring Boot applications, third-party library integrations.

Approach 1: XML-Based Container Configuration (`beans.xml`)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Define Bean for Country Repository -->
    <bean id="countryRepository" class="com.example.repository.CountryRepositoryImpl" />

    <!-- Define Bean for Country Service with Setter Injection -->
    <bean id="countryService" class="com.example.service.CountryServiceImpl">
        <property name="countryRepository" ref="countryRepository" />
    </bean>
</beans>
```

Approach 2: Java-Based Container Configuration (`AppConfig.java`)

```java
package com.example.config;

import com.example.repository.CountryRepository;
import com.example.repository.CountryRepositoryImpl;
import com.example.service.CountryServiceImpl;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {

    @Bean
    public CountryRepository countryRepository() {
        return new CountryRepositoryImpl();
    }

    @Bean
    public CountryServiceImpl countryService() {
        // Injecting dependency via Java Method Call
        return new CountryServiceImpl(countryRepository());
    }
}
```

Hands-on Container Bootstrapping

To initialize the container and retrieve a bean in application logic:

```java
public class IoCApplication {
    public static void main(String[] args) {
        // Bootstrapping Spring IoC via Java Configuration
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);

        CountryServiceImpl service = context.getBean(CountryServiceImpl.class);
        System.out.println("IoC Container successfully initialized! Bean: " + service);
    }
}
```

Exercise 7: Implementing Constructor and Setter Injection

Theory: Constructor vs. Setter Injection

Constructor Injection: Dependencies are provided through the class constructor. Guarantees that the object is always instantiated in a valid, fully-initialized state. Promotes immutability by allowing fields to be marked as `final`. Preferred by the Spring team for mandatory dependencies.

Setter Injection: Dependencies are assigned via JavaBeans-style setter methods. Useful for optional dependencies or parameters that can be dynamic/reconfigured during runtime. Leaves objects vulnerable to NullPointerExceptions if invoked prior to dependency assignment.

Constructor Injection Implementation

Below is the standard enterprise pattern for Constructor Injection using Spring annotations and `final` fields:

```
package com.example.service;

import com.example.model.Country;
import com.example.repository.CountryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service("countryConstructorService")
public class CountryServiceConstructorImpl {

    // Final fields guarantee immutability after construction
    private final CountryRepository countryRepository;

    // @Autowired is optional in Spring 4.3+ when a single constructor exists
    @Autowired
    public CountryServiceConstructorImpl(CountryRepository countryRepository) {
        if (countryRepository == null) {
            throw new IllegalArgumentException("CountryRepository must not be null");
        }
        this.countryRepository = countryRepository;
    }

    public Country getCountryByCode(String code) {
        return countryRepository.findByCode(code);
    }
}
```

Setter Injection Implementation

Below is the implementation using Setter Injection for optional or reconfigurable dependencies:

```java
package com.example.service;

import com.example.model.Country;
import com.example.repository.CountryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service("countrySetterService")
public class CountryServiceSetterImpl {

    private CountryRepository countryRepository;

    public CountryServiceSetterImpl() {
        // Default no-arg constructor required for Setter Injection
    }

    @Autowired
    public void setCountryRepository(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    public Country getCountryByCode(String code) {
        if (this.countryRepository == null) {
            throw new IllegalStateException("CountryRepository has not been injected.");
        }
        return countryRepository.findByCode(code);
    }
}
```

Module 2: Building Spring Boot Applications & Country Management REST API

Theory: Spring Boot Principles & Auto-Configuration

Spring Boot simplifies Spring development by providing **opinionated starter dependencies** and **Auto-Configuration**. Auto-configuration inspects classpath dependencies (e.g., presence of Tomcat or JPA drivers) and automatically configures relevant Spring beans unless overridden.

The core entry annotation `@SpringBootApplication` is a meta-annotation composed of:

- `@SpringBootConfiguration`: Designates the class as a primary source of bean definitions.
- `@EnableAutoConfiguration`: Triggers Spring Boot's auto-configuration mechanism.
- `@ComponentScan`: Scans the current package and sub-packages for Spring components (`@Service`, `@Repository`, `@RestController`).

Exercise 9: Creating a Spring Boot Application

1. Maven Build Specification (`pom.xml`)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.3</version>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>country-management-service</artifactId>
    <version>1.0.0</version>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Web Starter for RESTful Web Services -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <!-- Data JPA for Persistence -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <!-- H2 In-Memory Database -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>
</project>
```

2. Application Configuration (`application.yml`)

```yaml
server:
  port: 8080

spring:
  application:
    name: country-service
  datasource:
    url: jdbc:h2:mem:countrydbDB;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
    username: sa
    password:
  h2:
    console:
      enabled: true
      path: /h2-console
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate.format_sql: true
```

3. Spring Boot Main Application Class

```java
package com.example.country;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CountryManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(CountryManagementApplication.class, args);
    }
}
```

Implement Services for Managing Country

Requirement: Implement RESTful services to (1) Find a country based on country code, and (2) Add a new country.

1. Data Transfer Objects (DTOs) & Exception Handling

```java
package com.example.country.dto;

public record CountryDTO(
    String code,
    String name,
    String capitalName,
    Long population,
    String regionCode
) {}

// Custom Exception for missing resources
package com.example.country.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

2. Service Layer (`CountryService`)

```
package com.example.country.service;

import com.example.country.dto.CountryDTO;
import com.example.country.entity.Country;
import com.example.country.exception.ResourceNotFoundException;
import com.example.country.repository.CountryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class CountryServiceImpl {

    private final CountryRepository countryRepository;

    public CountryServiceImpl(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    /**
     * Requirement 1: Find a country based on country code
     */
    @Transactional(readOnly = true)
    public CountryDTO findCountryByCode(String code) {
        Country entity = countryRepository.findByCode(code)
                .orElseThrow(() -> new ResourceNotFoundException("Country not found with code: " + code));

        String capitalStr = (entity.getCapital() != null) ? entity.getCapital().getName() : "N/A";
        return new CountryDTO(entity.getCode(), entity.getName(), capitalStr, entity.getPopulation(),
entity.getRegionCode());
    }

    /**
     * Requirement 2: Add a new country
     */
    @Transactional
    public CountryDTO addNewCountry(CountryDTO dto) {
        if (countryRepository.existsById(dto.code())) {
            throw new IllegalArgumentException("Country with code " + dto.code() + " already exists!");
        }

        Country country = new Country();
        country.setCode(dto.code());
        country.setName(dto.name());
        country.setPopulation(dto.population());
        country.setRegionCode(dto.regionCode());

        Country saved = countryRepository.save(country);
        return new CountryDTO(saved.getCode(), saved.getName(), "N/A", saved.getPopulation(),
saved.getRegionCode());
    }
}
```

3. REST Controller Layer (`CountryRestController`)

```java
package com.example.country.controller;

import com.example.country.dto.CountryDTO;
import com.example.country.service.CountryServiceImpl;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/countries")
public class CountryRestController {

    private final CountryServiceImpl countryService;

    public CountryRestController(CountryServiceImpl countryService) {
        this.countryService = countryService;
    }

    // Endpoint 1: Find country based on country code
    @GetMapping("/{code}")
    public ResponseEntity<CountryDTO> getCountryByCode(@PathVariable("code") String code) {
        CountryDTO country = countryService.findCountryByCode(code);
        return ResponseEntity.ok(country);
    }

    // Endpoint 2: Add a new country
    @PostMapping
    public ResponseEntity<CountryDTO> addCountry(@RequestBody CountryDTO countryDTO) {
        CountryDTO created = countryService.addNewCountry(countryDTO);
        return new ResponseEntity<>(created, HttpStatus.CREATED);
    }
}
```

Module 3: Object-Relational Mapping (ORM) & Spring Data JPA

Theory: O/R Mapping Architecture & Domain Modeling

Object-Relational Mapping (ORM) bridges the relational model (tables, foreign keys, SQL types) with the Object-Oriented model (classes, instances, references, inheritance). JPA (Java Persistence API) is the standard Java specification, while Hibernate is the underlying persistence provider.

Key mapping elements demonstrate enterprise relational patterns:

- `@OneToOne`: `Country` <--> `Capital` (One capital per country).
- `@OneToMany` / `@ManyToOne`: `Country` (1) <--> (N) `City` (One country has multiple cities).

Demonstrate Implementation of O/R Mapping

1. `Capital` Entity (`@OneToOne` Relationship)

```java
package com.example.country.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "capitals")
public class Capital {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "capital_name", nullable = false, length = 100)
    private String name;

    @OneToOne(mappedBy = "capital")
    private Country country;

    // Constructors, Getters & Setters
    public Capital() {}
    public Capital(String name) { this.name = name; }
    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

2. `City` Entity (`@ManyToOne` Relationship)

```java
package com.example.country.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "cities")
public class City {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "country_code", nullable = false)
    private Country country;

    public City() {}
    public City(String name, Country country) { this.name = name; this.country = country; }
    public Long getId() { return id; }
    public String getName() { return name; }
    public Country getCountry() { return country; }
}
```

3. Primary Root Entity: `Country` Entity

```java
package com.example.country.entity;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "countries")
public class Country {

    @Id
    @Column(name = "country_code", length = 10, nullable = false)
    private String code;

    @Column(name = "country_name", nullable = false, length = 100)
    private String name;

    @Column(name = "population")
    private Long population;

    @Column(name = "region_code", length = 20)
    private String regionCode;

    // One-to-One mapping with Capital
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "capital_id", referencedColumnName = "id")
    private Capital capital;

    // One-to-Many mapping with City
    @OneToMany(mappedBy = "country", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<City> cities = new ArrayList<>();

    public Country() {}

    // Helper methods for bidirectional consistency
    public void addCity(City city) {
        cities.add(city);
    }

    // Standard Getters and Setters
    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Long getPopulation() { return population; }
    public void setPopulation(Long population) { this.population = population; }
    public String getRegionCode() { return regionCode; }
    public void setRegionCode(String regionCode) { this.regionCode = regionCode; }
    public Capital getCapital() { return capital; }
    public void setCapital(Capital capital) { this.capital = capital; }
}
```

Demonstrate Implementation of Query Methods Feature of Spring Data JPA

Theory: Derived Query Methods Mechanism

Spring Data JPA parses repository method names (e.g., `findByNameContainingIgnoreCase`) into target JPQL queries using a prefix like `find...By`, `read...By`, `query...By` or `count...By`. Criteria keywords like `And`, `Or`, `GreaterThan`, `LessThan`, `Between`, and `Like` dynamically build predicates without boiler-plate JPQL.

```java
package com.example.country.repository;

import com.example.country.entity.Country;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

    // 1. Derived Query Method: Find country by country code
    Optional<Country> findByCode(String code);

    // 2. Derived Query Method: Partial name matching (Case insensitive)
    List<Country> findByNameContainingIgnoreCase(String name);

    // 3. Derived Query Method: Find countries by population threshold
    List<Country> findByPopulationGreaterThanEqual(Long minPopulation);

    // 4. Derived Query Method: Multi-field filtering (Region and Population Range)
    List<Country> findByRegionCodeAndPopulationBetween(String regionCode, Long minPop, Long maxPop);

    // 5. Derived Query Method with Ordering
    List<Country> findByRegionCodeOrderByNameAsc(String regionCode);
}
```

Module 4: Advanced Persistence with Hibernate (HQL & Native Query)

Theory: HQL / JPQL vs Native SQL

Hibernate Query Language (HQL) / JPQL: An object-oriented query language that targets entity domain models (classes and properties) rather than underlying database tables and columns. HQL features portable database independence, entity auto-polymorphism, and managed state mapping.

Native SQL Queries: Raw database SQL queries executed directly against the database engine. Necessary when leveraging database-specific SQL features (e.g., JSON syntax, window functions, CTEs) or executing fine-tuned bulk execution plans.

Demonstrate Writing Hibernate Query Language (HQL) and Native Query

Below is a custom DAO repository implementation utilizing JPA `EntityManager` to execute pure HQL and Native Queries:

```java
package com.example.country.repository;

import com.example.country.entity.Country;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import jakarta.persistence.TypedQuery;
import jakarta.persistence.Query;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public class CountryCustomRepositoryImpl {

    @PersistenceContext
    private EntityManager entityManager;

    /**
     * DEMO 1: Hibernate Query Language (HQL) / JPQL - JOIN FETCH Query
     * Solves N+1 query problem by fetching related entity eagerly in single SELECT
     */
    public List<Country> findCountriesWithCapitalByRegionHQL(String regionCode) {
        String hql = "SELECT c FROM Country c JOIN FETCH c.capital WHERE c.regionCode = :region";
        TypedQuery<Country> query = entityManager.createQuery(hql, Country.class);
        query.setParameter("region", regionCode);
        return query.getResultList();
    }

    /**
     * DEMO 2: HQL Aggregate / Dynamic Projection Query
     */
    public Long getTotalPopulationByRegionHQL(String regionCode) {
        String hql = "SELECT SUM(c.population) FROM Country c WHERE c.regionCode = :region";
        TypedQuery<Long> query = entityManager.createQuery(hql, Long.class);
        query.setParameter("region", regionCode);
        return query.getSingleResult();
    }

    /**
     * DEMO 3: Native Query Mapping directly to Managed Entity
     */
    @SuppressWarnings("unchecked")
    public List<Country> findCountriesByNativeSQL(String regionCode) {
        String nativeSql = "SELECT * FROM countries WHERE region_code = :reg AND population > :minPop";
        Query query = entityManager.createNativeQuery(nativeSql, Country.class);
        query.setParameter("reg", regionCode);
        query.setParameter("minPop", 1000000L);
        return query.getResultList();
    }

    /**
     * DEMO 4: Spring Data @Query Annotation Example (HQL & Native SQL)
     */
    // Interface-based Spring Data Query annotations:
    // @Query("SELECT c FROM Country c WHERE c.code = :code")  <-- HQL
    // @Query(value = "SELECT * FROM countries WHERE country_code = :code", nativeQuery = true) <-- Native
}
```

Module 5: Spring Cloud Microservices — Eureka Discovery Server & Registration

> **Theory: Microservices Architecture & Service Discovery**

In distributed microservice architectures, service instances dynamically scale up, down, or experience network location shifts (IP addresses / dynamic ports). Hardcoding microservice endpoints leads to fragile systems.

Netflix Eureka serves as a Service Registry where microservices automatically register their metadata (IP, port, health status) upon boot. Clients fetch registry info and perform client-side load balancing via dynamic routing.

Create Eureka Discovery Server and Register Microservices

Step 1: Create Netflix Eureka Discovery Server Application

1. Dependencies (`pom.xml` for Eureka Server):

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

2. Configuration (`application.yml` for Eureka Server):

```
server:
  port: 8761

spring:
  application:
    name: eureka-server

eureka:
  instance:
    hostname: localhost
  client:
    # Eureka server does not need to register with itself
    register-with-eureka: false
    fetch-registry: false
    service-url:
      defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/
```

3. Application Class (`EurekaServerApplication.java`):

```
package com.example.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer // Activates Netflix Eureka Server Registry
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Step 2: Register Microservices (Country Service) with Eureka Server

1. Add Client Dependency to Country Microservice (`pom.xml`):

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

2. Configure Eureka Client Properties (`application.yml` in Country Microservice):

```
server:
  port: 8081

spring:
  application:
    name: country-service # Registered Service ID in Eureka Registry

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    instance-id: ${spring.application.name}:${random.value}
```

3. Enable Eureka Discovery in Application Boot Class:

```
package com.example.country;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient // Registers instance with Eureka Discovery Server
public class CountryManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(CountryManagementApplication.class, args);
    }
}
```

> **Verification & Eureka Dashboard Inspection**
>
> 1. Start **Eureka Discovery Server** (`EurekaServerApplication`) on Port 8761.
> 2. Launch **Country Microservice** (`CountryManagementApplication`) on Port 8081.
> 3. Access the Eureka Dashboard via Web Browser at `http://localhost:8761`.
> 4. Verify that **COUNTRY-SERVICE** appears under the *"Instances currently registered with Eureka"* table with Status **UP (1) - 127.0.0.1:country-service:8081**.